

Coffee Shop Data Platform

Data Engineering Project Report

Github Repo

{https://github.com/Khangelani-Mnguni/coffee_shop_sales_project}

Lookerstudio Dash

{<https://datastudio.google.com/reporting/93fd9c11-a81b-437c-bf4f-10cb813523b0>}

Executive Summary

This report details the architectural design and technical implementation of a modern, cloud-native Extract, Load, and Transform (ELT) data platform tailored for a multi-store coffee shop enterprise.

The primary objective was to architect and deploy a highly scalable, automated analytics ecosystem. This platform is engineered to ingest raw transactional sales data, seamlessly transform it into pristine, analytics-ready datasets, and empower the business with actionable insights through comprehensive reporting and machine learning-optimized data marts.

The enterprise technology stack leverages the following robust tools and services:

- Google Cloud Platform (GCP) for core infrastructure
- Google BigQuery as the enterprise data warehouse
- Terraform for Infrastructure as Code (IaC)
- Python for custom ingestion frameworks
- dbt (data build tool) for data transformation
- Apache Airflow for workflow orchestration
- GitHub Actions for CI/CD automation
- Looker Studio for business intelligence and visualization

The delivered platform rigorously adheres to modern Data Engineering best practices, ensuring environment isolation, automated testing, and highly repeatable deployments.

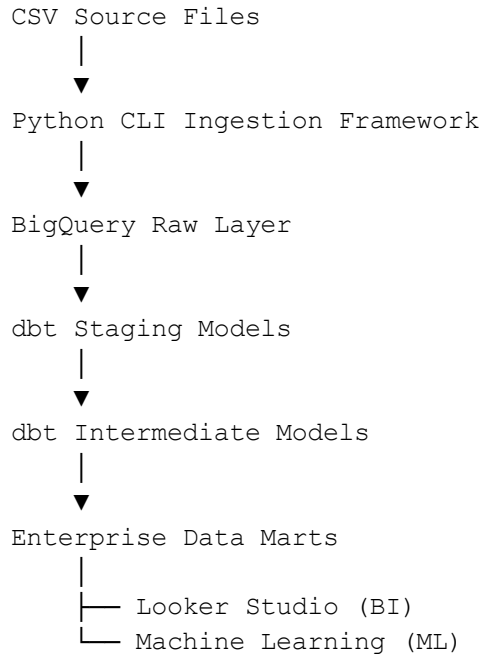
Project Objectives

To ensure maximum business value and technical reliability, the following core objectives were established:

- Architect an end-to-end automated ELT pipeline.
- Centralize diverse data streams within a highly available cloud data warehouse.
- Manage and provision all cloud resources via Infrastructure as Code (Terraform).
- Develop modular, version-controlled, and documented transformation logic using dbt.
- Enforce data integrity through comprehensive data quality testing.
- Streamline delivery pipelines utilizing GitHub Actions for CI/CD.
- Implement robust DAG-based workflow orchestration with Apache Airflow.
- Produce reliable, performant datasets optimized for BI reporting.
- Expose specialized data marts structured for advanced machine learning applications.

Solution Architecture

The system employs a multi-tiered ELT architectural pattern, ensuring separation of concerns between ingestion, storage, and presentation layers.



Infrastructure Provisioning (IaC)

All underlying infrastructure is codified and managed using Terraform. This approach guarantees stateful, version-controlled, and auditable resource provisioning across all environments.

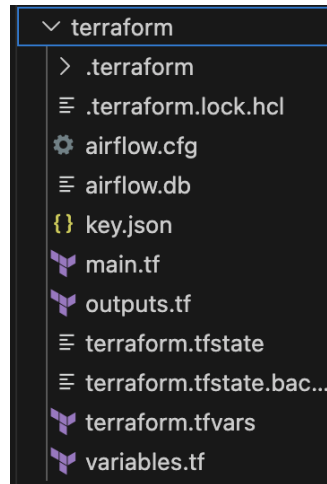
Key advantages achieved through Terraform include:

- 100% reproducible infrastructure environments.
- Source-controlled configuration management.
- Elimination of manual configuration drift.
- Seamless scalability for future architectural expansions.

The following BigQuery datasets are formally managed via Terraform state:

```
coffee_raw  
coffee_dev  
coffee_prod
```

Figure 2: Terraform Project Structure



Data Ingestion Layer

Raw source files are seamlessly ingested utilizing a custom-built Python Command Line Interface (CLI) application. This utility validates and loads flat files directly into the BigQuery raw tables.

Identified Source Feeds:

```
Astoria.csv
Hells_Kitchen.csv
Lower_Manhattan.csv
```

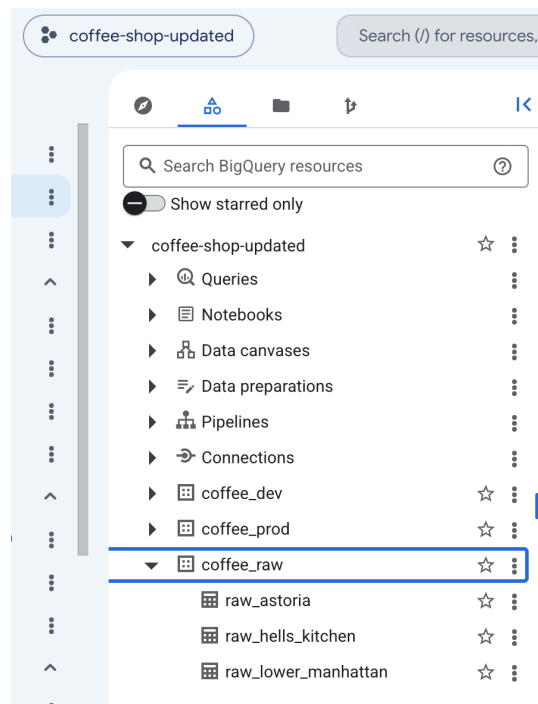
These files are mapped into isolated tables within the raw dataset:

```
coffee_raw
├── raw_astoria
├── raw_hells_kitchen
└── raw_lower_manhattan
```

Figure 3: Python Ingestion Framework Execution

```
cli.py x
ingestion > cli.py > ...
MAC, last month | 1 author (MAC)
1 import click
2 import yaml
3 from loader import BigQueryLoader
4
5
6 @click.command()
7 @click.option("--file", required=True, help="Path to CSV file")
8 @click.option("--store", required=True, help="Store name")
9 @click.option("--mode", default=None, help="append or truncate")
10 def load(file, store, mode):
11
12     with open("ingestion/config.yaml") as f:
13         config = yaml.safe_load(f)
14
15     write_mode = mode or config["write_mode"]
16
17     loader = BigQueryLoader(
18         config["project_id"],
19         config["dataset"],
20         write_mode
21     )
22
23     table_name = f"raw_{store.lower().replace(' ', '_')}"
24
25     loader.load_csv(file, table_name, store)
26
27
28 if __name__ == "__main__":
29     load()
```

Figure 4: Raw Dataset Representation in BigQuery



Cloud Data Warehouse (BigQuery)

Google BigQuery functions as the central analytics engine. The data model is strictly segregated into three logical zones:

Raw Layer

Acts as an immutable operational data store, housing unmodified source records.

```
coffee_raw
```

Development Layer

An isolated sandbox environment enabling data engineers to build and validate dbt models locally without impacting production workloads.

```
coffee_dev
```

Production Layer

The highly-governed, final presentation layer containing sanitized, trusted enterprise datasets.

```
coffee_prod
```

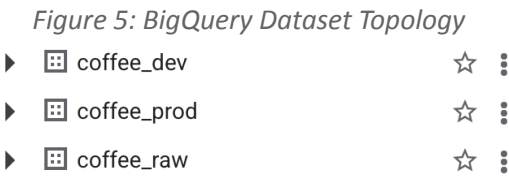


Figure 6: BigQuery Relational Tables

▼ ☐ coffee_dev	☆ ⋮
☐ calendar	☆ ⋮
☐ int_combined_stores	☆ ⋮
☐ int_sales	☆ ⋮
☐ mart_bi	☆ ⋮
☐ mart_ml	☆ ⋮
☐ staging_astoria	☆ ⋮
☐ staging_hells_kitchen	☆ ⋮
☐ staging_lower_manhattan	☆ ⋮
▼ ☐ coffee_prod	☆ ⋮
☐ calendar	☆ ⋮
☐ int_combined_stores	☆ ⋮
☐ int_sales	☆ ⋮
☐ mart_bi	☆ ⋮
☐ mart_ml	☆ ⋮
☐ staging_astoria	☆ ⋮
☐ staging_hells_kitchen	☆ ⋮
☐ staging_lower_manhattan	☆ ⋮
▼ ☐ coffee_raw	☆ ⋮
☐ raw_astoria	☆ ⋮
☐ raw_hells_kitchen	☆ ⋮

Data Transformation Engine (dbt)

dbt (data build tool) orchestrates all in-warehouse transformations, compiling SQL business logic into scalable materializations.

Core transformation methodologies implemented:

- Timestamp and temporal standardization.
- Strict data type enforcement and casting.
- Store location name mapping.
- Record deduplication and anomaly filtration.
- Implementation of complex business metric logic.
- Continuous data quality validation.

Data Quality Assurance

Automated assertions are executed against the pipeline. Key tests include:

```
Field: transaction_id
Constraint: NOT NULL & UNIQUE
```

Key Transformations

Automated store mapping for local reporting:

Field: store_location

Transformation: Astoria → West Campus, Hells Kitchen → East Campus, Lower Manhattan → Main Campus

Transformation Lineage

Sources → Staging → Intermediate → Marts

Figure 7: dbt Project Organization

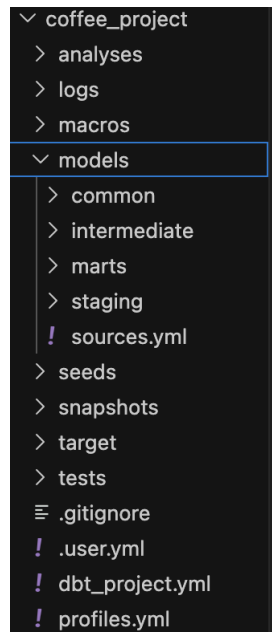


Figure 8: Successful dbt Compilation and Execution

```
(.venv) (base) mac0192 coffee_project % dbt build --select staging
19:42:55 Running with dbt=1.11.8
19:42:57 Registered adapter: bigquery=1.11.1
19:42:58 [WARNING][CustomKeyInObjectDeprecation]: Deprecated functionality
Custom key 'columns' found at 'models[0].columns[14]' in file
'models/staging/staging.yml'. This may mean the key is a typo, or is simply not
a key supported by the object.
19:42:58 [WARNING]: Configuration paths exist in your dbt_project.yml file which do not apply to any resource
s.
There are 1 unused configuration paths:
- models.coffee_project.example
19:42:58 Found 8 models, 2 data tests, 3 sources, 539 macros
19:42:58 Concurrency: 4 threads (target='dev')
19:42:58
19:43:02 1 of 5 START sql view model coffee_dev.staging_astoria ..... [RUN]
19:43:02 2 of 5 START sql view model coffee_dev.staging_hells_kitchen ..... [RUN]
19:43:02 3 of 5 START sql view model coffee_dev.staging_lower_manhattan ..... [RUN]
19:43:05 3 of 5 OK created sql view model coffee_dev.staging_lower_manhattan ..... [CREATE VIEW (0 pro
cessed) in 2.31s]
19:43:05 2 of 5 OK created sql view model coffee_dev.staging_hells_kitchen ..... [CREATE VIEW (0 pro
cessed) in 2.52s]
19:43:07 1 of 5 OK created sql view model coffee_dev.staging_astoria ..... [CREATE VIEW (0 pro
cessed) in 4.15s]
19:43:07 4 of 5 START test not_null_staging_astoria_transaction_id ..... [RUN]
19:43:07 5 of 5 START test unique_staging_astoria_transaction_id ..... [RUN]
19:43:09 4 of 5 PASS not_null_staging_astoria_transaction_id ..... [PASS in 2.58s]
19:43:10 5 of 5 PASS unique_staging_astoria_transaction_id ..... [PASS in 2.98s]
19:43:10
19:43:10 Finished running 2 data tests, 3 view models in 0 hours 0 minutes and 11.72 seconds (11.72s).
19:43:10
19:43:10 Completed successfully
19:43:10
19:43:10 Done. PASS=5 WARN=0 ERROR=0 SKIP=0 NO-OP=0 TOTAL=5
19:43:10 [WARNING][DeprecationsSummary]: Deprecated functionality
Summary of encountered deprecations:
- CustomKeyInObjectDeprecation: 2 occurrences
To see all deprecation instances instead of just the first occurrence of each,
run command again with the '--show-all-deprecations' flag. You may also need to
run with '--no-partial-parse' as some deprecations are only encountered during
parsing.
(.venv) (base) mac0192 coffee_project %
```


Figure 9: dbt Auto-Generated Documentation and Lineage Graph



Enterprise Data Marts

The apex of the transformation pipeline yields optimized data marts designed for specific downstream consumption patterns.

Business Intelligence Mart (mart_bi)

Purpose-built for analytical reporting, dynamic dashboards, and executive KPI tracking.

Machine Learning Mart (mart_ml)

Denormalized, feature-rich datasets engineered for predictive modeling and advanced statistical analysis.

Figure 10: mart_ml Query Validation

The screenshot shows a query validation interface. At the top, it displays the query: `1 SELECT * FROM 'coffee-shop-updated.coffee_dev.mart_ml' LIMIT 1000`. Below the query, it indicates 'Query completed' and 'Using on-demand processing quota'. The main section is titled 'Query results' and contains a table with 14 rows of data. The table has columns for 'transaction_id', 'transaction_date', 'store_location', 'product_category', 'product_detail', and 'product_type'. The data shows various transactions from different locations and categories.

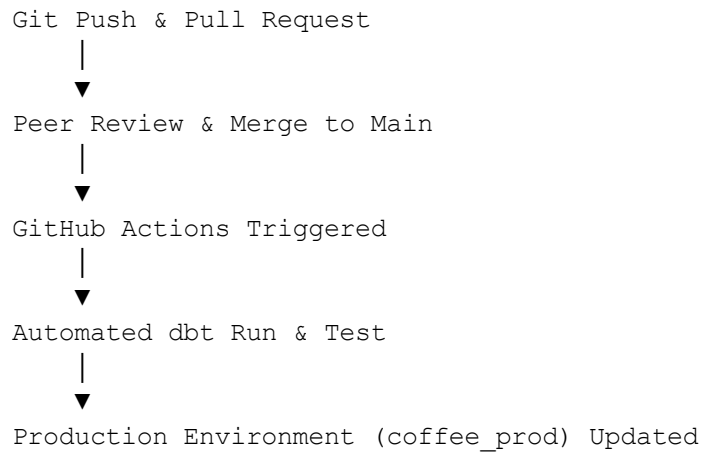
row	transaction_id	transaction_date	store_location	product_category	product_detail	product_type
1	514	2025-01-01	East Campus	Bakery	Almond Croissant	Pastry
2	103	2025-01-01	East Campus	Bakery	Almond Croissant	Pastry
3	274	2025-01-01	Main Campus	Coffee	Brazilian Lg	Organic brewed
4	523	2025-01-01	West Campus	Coffee	Brazilian Lg	Organic brewed
5	249	2025-01-01	East Campus	Coffee	Brazilian Lg	Organic brewed
6	506	2025-01-01	West Campus	Coffee	Brazilian Lg	Organic brewed
7	538	2025-01-01	East Campus	Coffee	Brazilian Lg	Organic brewed
8	391	2025-01-01	West Campus	Coffee	Brazilian Lg	Organic brewed
9	520	2025-01-01	East Campus	Coffee	Brazilian Lg	Organic brewed
10	418	2025-01-01	West Campus	Coffee	Brazilian Lg	Organic brewed
11	452	2025-01-01	West Campus	Coffee	Brazilian Lg	Organic brewed
12	164	2025-01-01	East Campus	Coffee	Brazilian Lg	Organic brewed
13	33	2025-01-01	Main Campus	Coffee	Brazilian Lg	Organic brewed
14	455	2025-01-01	East Campus	Coffee	Brazilian Lg	Organic brewed

CI/CD and Automation

A strict Continuous Integration and Continuous Deployment (CI/CD) framework was engineered using GitHub Actions to govern release cycles securely.

Developer Feature Branch

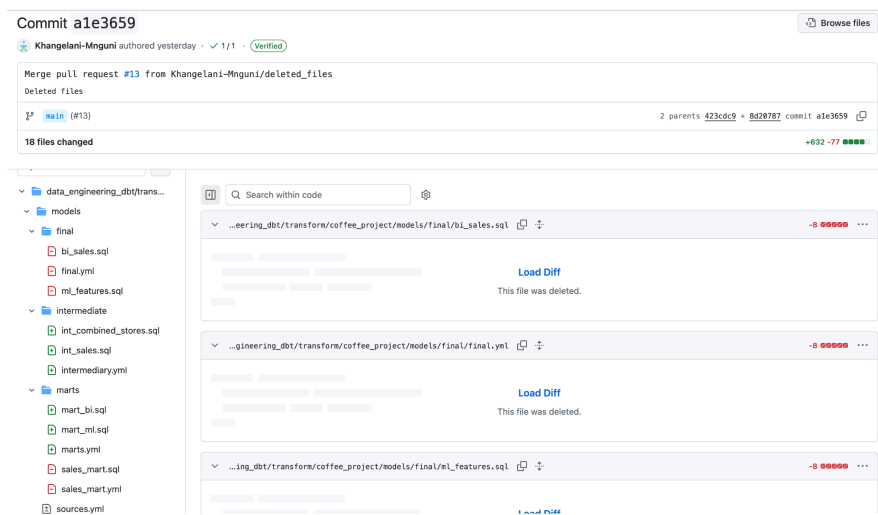


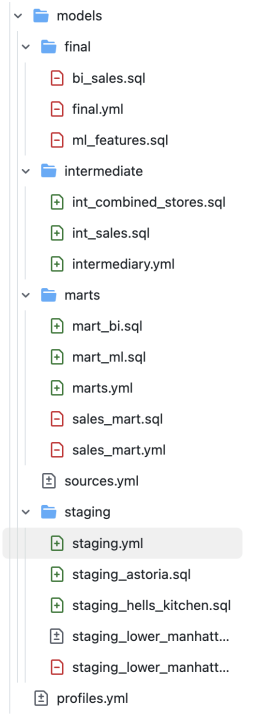


Business Benefits:

- Zero-downtime, automated deployment pipelines.
- Drastic reduction in manual release overhead.
- Enforced consistency and auditability across production releases.

Figure 11: GitHub Actions CI/CD Pipeline Execution





Workflow Orchestration

Apache Airflow serves as the central command node, orchestrating the temporal execution of the entire ELT lifecycle.

Standardized Daily Batch Schedule:

1. Initialize DAG
2. Trigger Python Ingestion (Load New Data)
3. Execute dbt Transformation Models
4. Run dbt Data Quality Tests
5. Publish Updates to coffee_prod
6. Trigger BI Dashboard Refresh
7. Terminate DAG Successfully

Figure 12: Airflow Directed Acyclic Graph (DAG)

```
# -----
# DAG
# -----

with DAG(
    dag_id="coffee_pipeline",
    default_args=default_args,
    schedule_interval="@daily",
    catchup=True,
    max_active_runs=1,
) as dag:

    # 1. Detect files
    detect_files = PythonOperator(
        task_id="detect_files",
        python_callable=list_csv_files,
    )

    # 2. Ingest data
    ingest = PythonOperator(
        task_id="ingest_data",
        python_callable=process_file,
    )

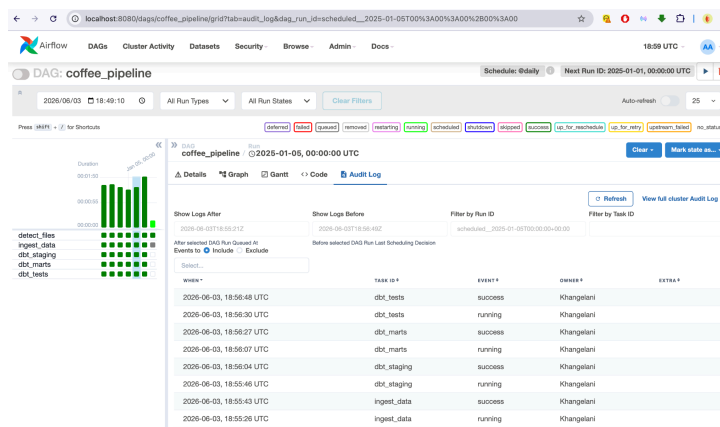
    # 3. dbt staging
    dbt_staging = BashOperator(
        task_id="dbt_staging",
        bash_command="""
        cd /opt/airflow/project/transform/coffee_project &&
        dbt run --select staging --target prod
        """
    )

    # 4. dbt marts
    dbt_marts = BashOperator(
        task_id="dbt_marts",
        bash_command="""
        cd /opt/airflow/project/transform/coffee_project &&
        dbt run --select marts --target prod
        """
    )

    # 5. dbt tests
    dbt_tests = BashOperator(
        task_id="dbt_tests",
        bash_command="""
        cd /opt/airflow/project/transform/coffee_project &&
        dbt test --target prod
        """
    )

    # FLOW
    detect_files >> ingest >> dbt_staging >> dbt_marts >> dbt_tests
```

Figure 13: Successful Airflow Task Execution Log
Details

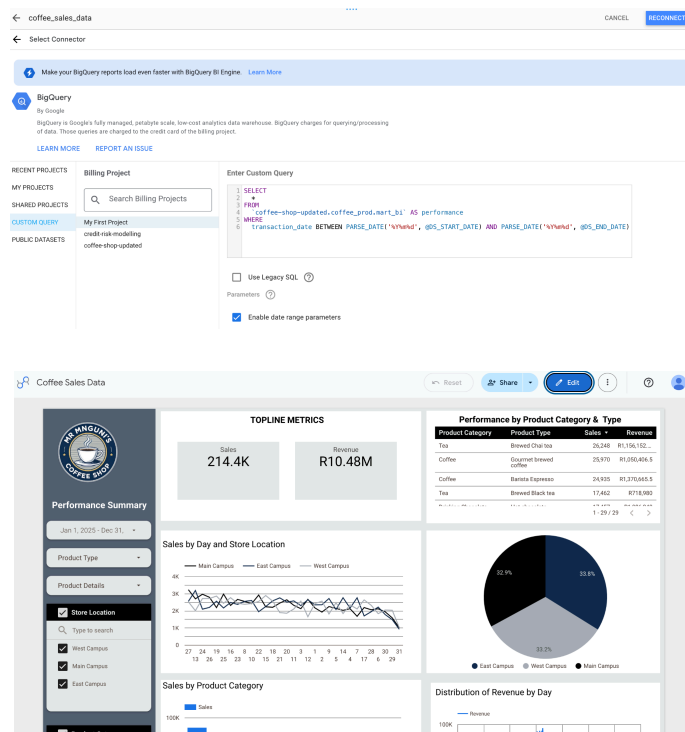


Business Intelligence & Executive Reporting

Looker Studio provides the interactive presentation layer, interfacing directly with the coffee_prod.mart_bi view. The executive dashboard surfaces critical operational intelligence, including:

- Aggregate and granular revenue analysis.
- Product lifecycle and performance tracking.
- Cross-store comparative performance.
- Temporal transaction volume trends.
- Core operational KPIs.

Figure 14: Looker Studio Executive Dashboard



Data-Driven Business Insights

The platform immediately empowers stakeholders to answer critical operational queries using standardized SQL over the data marts.

1. Store Performance Benchmarking

Query: Which location generated the highest aggregate revenue?

Business Value: Identifies the flagship location, driving decisions around capital allocation and successful operational modeling.

Figure 15: Top Revenue Store Query Results

1

SELECT store_location, SUM(total_revenue) AS Revenue FROM `coffee-shop-updated.coffee\_prod.mart\_bi` GROUP BY 1 ORDER BY Revenue DESC

✓

Query completed

Using on-demand processing quota

Query results

Chat

Save results

Open in

Job information

Results

Visualization

JSON

Execution details

Execution graph

Row	store_location	Revenue	
1	East Campus	3547294.050000...	
2	West Campus	3483448.650000...	
3	Main Campus	3450339.75	

2. Temporal Revenue Peaks

Query: What are the top 5 revenue-generating days in the dataset?

Business Value: Isolates peak trading periods to optimize supply chain logistics and seasonal staffing models.

Figure 16: Top Revenue Days Query Results

1

SELECT transaction_date, SUM(total_revenue) AS Revenue FROM `coffee-shop-updated.coffee\_prod.mart\_bi` GROUP BY 1 ORDER BY Revenue DESC LIMIT 5

✔ Query completed

Using on-demand processing quota

Query results

🗨 Chat

Save results ▾

Open in ▾

Job information

Results

Visualization

JSON

Execution details

Execution graph

Row	transaction_date ▾	Revenue ▾
1	2025-06-19	96058.65
2	2025-06-13	92840.4
3	2025-06-08	92197.35
4	2025-06-17	91764.0
5	2025-06-18	90391.35

3. Product Profitability

Query: Which specific SKUs drive the highest total revenue?

Business Value: Directly informs inventory purchasing strategies, menu optimization, and targeted promotional campaigns.

Figure 17: Top Products Query Results

1		SELECT product_category, SUM(total_revenue) AS Revenue FROM `coffee-shop-updated.coffee_prod.mart_bi` GROUP BY 1 ORDER BY Revenue DESC LIMIT 5	
Query completed		Using on-demand processing quota	
Query results		Chat Save results Open in	
Job information		Results Visualization JSON Execution details Execution graph	
Row	product_category	Revenue	
1	Coffee	4048389.75	
2	Tea	2946012.75	
3	Bakery	1234629.599999...	
4	Drinking Chocolate	1086240.0	
5	Coffee beans	601278.75	

4. Trading Day Optimization

Query: Which day of the week historically generates the maximum sales volume?

Business Value: Enables precision scheduling of store associates and optimized daily operational workflows.

Figure 18: Best Trading Day Query Results

1		SELECT day_of_week, SUM(total_revenue) AS Revenue FROM `coffee-shop-updated.coffee_prod.mart_bi` GROUP BY 1 ORDER BY Revenue DESC	
Query completed		Using on-demand processing quota	
Query results		Chat Save results Open in	
Job information		Results Visualization JSON Execution details Execution graph	
Row	day_of_week	Revenue	
1	5	1524739.200000...	
2	2	1520595.000000...	
3	1	1511416.200000...	
4	7	1504703.1	
5	6	1491434.100000...	
6	4	1474851.150000...	
7	3	1453343.700000...	

5. Consumer Behavior Metrics

Query: Which store records the highest average transaction value (ATV)?

Business Value: Highlights regional consumer spending habits, identifying opportunities for cross-selling and up-selling training.

Figure 19: Average Basket Value Query Results

The screenshot displays a SQL query interface. At the top, a SQL query is shown: `1 SELECT store_location, SUM(total_revenue/transaction_qty) AS Average_Transaction_Value FROM `coffee-shop-updated.coffee_prod.mart_bi` GROUP BY 1 ORDER BY Average_Transaction_Value DESC`. Below the query, a status bar indicates 'Query completed' and 'Using on-demand processing quota'. The main section, titled 'Query results', contains a table with three columns: 'Row', 'store_location', and 'Average_Transac...'. The table has three rows of data. Navigation links for 'Chat', 'Save results', and 'Open in' are visible to the right of the table header.

Row	store_location	Average_Transac...
1	East Campus	2582898.300000...
2	West Campus	2568981.150000...
3	Main Campus	2412643.499999...

Future Analytical Opportunities

The foundational data model natively supports advanced future explorations, including:

- Granular peak trading hour analysis by specific location.
- Deep cohort analysis of customer purchasing behavior.
- Macro-level product category performance tracking.
- Predictive revenue and demand forecasting.
- Market Basket Analysis (Product affinity and association rules).

Technical Challenges & Resolutions

Throughout the software development lifecycle, several engineering challenges were identified and systematically resolved:

Data Standardization Discrepancies

Challenge: Ingested source files from diverse store locations exhibited heterogeneous schemas and data formatting.

Resolution: Engineered resilient staging transformations within dbt to cast, clean, and standardize schemas prior to downstream integration.

Deployment Consistency

Challenge: Manual schema deployments risked environmental drift between development and production layers.

Resolution: Abstracted deployment workflows into automated GitHub Actions, ensuring hermetic, repeatable CI/CD pipelines.

Data Integrity Risks

Challenge: The potential for corrupted or duplicate transactional records to pollute executive reporting.

Resolution: Enforced strict generic and custom data tests within dbt to validate uniqueness, referential integrity, and non-null constraints at build time.

Project Outcomes and Conclusion

This project successfully architected and delivered a robust, enterprise-grade Data Engineering ecosystem. The finalized solution achieves its primary mandate by providing:

- A fully automated, low-latency ELT pipeline.
- A secure, highly scalable cloud-native architecture.
- Complete Infrastructure as Code (IaC) governance.
- Zero-touch automated deployments and task orchestration.
- Highly performant, reliable datasets for both BI and ML applications.

By leveraging industry-standard methodologies, this platform ensures strict data quality, operational efficiency, and a solid analytical foundation. The modular architecture is strategically designed to scale effortlessly, gracefully accommodating future business expansion and the integration of novel data sources with minimal engineering overhead.